

Annotated RCOOutput Backend

ArduPilot Rover skid steering with Raspberry Pi + PCA9685 + L298N

This PDF documents a prototype AP_HAL::RCOutput backend for a skid-steering Rover setup where the PCA9685 controls both L298N speed enable pins and direction pins. It is written as teaching/experimentation code, not as a drop-in patch guaranteed to compile unchanged on every ArduPilot branch.

Safety note

Bench-test with wheels off the ground first. Verify motor polarity, direction channels, stop behavior, and failsafe behavior before connecting a battery capable of high current.

1. Configuration overview

Layer	Role
Rover control / mixer	Computes left and right motor commands for skid steering.
SRV_Channel / SERVOx_FUNCTION	Maps logical functions to output channels: SERVO1 = left throttle, SERVO3 = right throttle.
Custom RCOOutput backend	Converts 1000-2000 us motor commands into PCA9685 duty cycles.
PCA9685	Generates PWM for L298N ENA/ENB and digital-like 0%/100% outputs for IN1-IN4.
L298N	Applies motor power and polarity to left/right brushed DC motors.

ArduPilot output	Internal channel	Function	Physical expansion
SERVO1	0	Throttle Left	PCA9685 CH0/CH1/CH2 -> ENA/IN1/IN2
SERVO3	2	Throttle Right	PCA9685 CH3/CH4/CH5 -> ENB/IN3/IN4

PCA9685 channel	L298N pin	Meaning
CH0	ENA	Left motor speed PWM
CH1	IN1	Left direction A
CH2	IN2	Left direction B
CH3	ENB	Right motor speed PWM
CH4	IN3	Right direction A
CH5	IN4	Right direction B

Why there is no explicit turn-left/turn-right function

RCOutput is deliberately dumb. ArduPilot Rover already performs the skid-steering mix above this layer. The backend receives one left motor value and one right motor value. If those values differ, the vehicle turns. If they are opposite, it spins.

2. Movement examples

Movement	SERVO1 / left	SERVO3 / right	Backend effect
Stop	1500	1500	Both EN duty = 0; direction pins off.
Forward	1700	1700	Both sides forward at about 40% duty.

Reverse	1300	1300	Both sides reverse at about 40% duty.
Gentle left	1450	1700	Left slow/reverse-side near stop, right forward.
Gentle right	1700	1450	Left forward, right near stop.
Spin left/right	1300	1700	Left reverse, right forward. Direction depends on wiring.
Spin opposite	1700	1300	Left forward, right reverse. Direction depends on wiring.

3. Design decisions

Decision	What	Why	How
Use SERVO1 and SERVO3	Default-style Rover skid-steer outputs.	Keeps the setup aligned with ArduPilot Rover conventions.	SERVO1_FUNCTION=73 and SERVO3_FUNCTION=74.
Use 1500 us as stop	Treat output as bidirectional throttle.	Same neutral concept as RC car/rover throttle.	Map 1000..1500..2000 us to -1..0..+1.
Use PCA9685 for direction too	No Pi GPIO direction pins.	Your wiring has PCA9685 connected to ENA/INx/ENB.	IN pins are 0% or 100% duty.
No steering logic in RCOOutput	Only hardware output code lives here.	Keeps HAL layer separate from vehicle logic.	write(channel, value) routes channel 0/2 to left/right motor.
Use cork/push	Batch updates.	Avoid one motor updating before the other.	Store values during cork(), write both on push().

4. Annotated header file: RCOOutput_PCA9685_L298N.h

```
#pragma once

#include <AP_HAL/AP_HAL.h>

namespace Linux {

// WHAT:
//   RCOOutput_PCA9685_L298N is a custom AP_HAL::RCOutput backend.
//
// WHY:
//   ArduPilot Rover outputs only two logical skid-steer commands:
//   - SERVO1 / channel 0 = left motor
//   - SERVO3 / channel 2 = right motor
//   But the physical L298N needs six control signals:
//   - ENA, IN1, IN2 for the left motor
//   - ENB, IN3, IN4 for the right motor
//   This backend expands the two ArduPilot motor commands into six PCA9685 outputs.
//
// HOW:
//   ArduPilot writes normal RC-style values such as 1000, 1500, 2000 us.
//   The backend interprets them as bidirectional motor commands:
//   - 1500 us = stop
//   - greater than 1500 us = forward
//   - less than 1500 us = reverse
//   It then sets direction pins to 0% or 100% duty and speed enable pins to
//   proportional PWM duty using the PCA9685.
class RCOOutput_PCA9685_L298N : public AP_HAL::RCOutput {
public:
    RCOOutput_PCA9685_L298N(AP_HAL::I2CDevice *dev,
                           uint8_t left_en,
                           uint8_t left_in1,
                           uint8_t left_in2,
                           uint8_t right_en,
                           uint8_t right_in1,
                           uint8_t right_in2);

    // Called once by the HAL during startup.
    void init() override;

    // ArduPilot may request an output frequency. PCA9685 has one shared
    // frequency for all 16 channels, so this backend clamps and applies one value.
    void set_freq(uint32_t chmask, uint16_t freq_hz) override;
    uint16_t get_freq(uint8_t chan) override;

    // Enables/disables logical ArduPilot output channels.
    void enable_ch(uint8_t chan) override;
    void disable_ch(uint8_t chan) override;
};

}
```

```

// Main output method. ArduPilot calls this with a logical channel number
// and a PWM-like period in microseconds.
void write(uint8_t chan, uint16_t period_us) override;

// Read back the last value written to each logical channel.
uint16_t read(uint8_t chan) override;
void read(uint16_t *period_us, uint8_t len) override;

// cork()/push() let ArduPilot batch updates so left and right motors update
// together. This reduces transient steering glitches.
void cork() override;
void push() override;

private:
// ArduPilot uses zero-based channel indexes internally:
//   channel 0 = SERV01
//   channel 2 = SERV03
static constexpr uint8_t LEFT_AP_CH = 0; // SERV01
static constexpr uint8_t RIGHT_AP_CH = 2; // SERV03

// Bidirectional RC-style command range.
static constexpr uint16_t PWM_MIN = 1000;
static constexpr uint16_t PWM_STOP = 1500;
static constexpr uint16_t PWM_MAX = 2000;
static constexpr uint16_t DEADZONE_US = 30;

// PCA9685 register addresses.
static constexpr uint8_t PCA9685_MODE1 = 0x00;
static constexpr uint8_t PCA9685_MODE2 = 0x01;
static constexpr uint8_t PCA9685_LED0_ON_L = 0x06;
static constexpr uint8_t PCA9685_PRESCALE = 0xFE;

AP_HAL::I2CDevice *_dev;

// Physical PCA9685 channel mapping.
uint8_t _left_en;
uint8_t _left_in1;
uint8_t _left_in2;
uint8_t _right_en;
uint8_t _right_in1;
uint8_t _right_in2;

// State for logical ArduPilot output channels.
uint16_t _last_pwm[16];
bool _enabled[16];

bool _corked;
bool _pending;
uint16_t _freq_hz;

void init_pca9685();
void set_pca9685_freq(uint16_t freq_hz);

void write_motor(bool left, uint16_t pwm_us);

void pca_write_channel(uint8_t channel, uint16_t on, uint16_t off);
void pca_write_duty(uint8_t channel, float duty);
void pca_on(uint8_t channel);
void pca_off(uint8_t channel);

static float pwm_to_speed(uint16_t pwm_us);
};

} // namespace Linux

```

5. Annotated source file: RCOOutput_PCA9685_L298N.cpp

```

#include "RCOutput_PCA9685_L298N.h"

#include <AP_HAL/AP_HAL.h>
#include <AP_Math/AP_Math.h>

#include <math.h>
#include <string.h>

using namespace Linux;

extern const AP_HAL::HAL& hal;

RCOutput_PCA9685_L298N::RCOutput_PCA9685_L298N(
    AP_HAL::I2CDevice *dev,
    uint8_t left_en,
    uint8_t left_in1,
    uint8_t left_in2,
    uint8_t right_en,

```

```

    uint8_t right_in1,
    uint8_t right_in2
) :
    _dev(dev),
    _left_en(left_en),
    _left_in1(left_in1),
    _left_in2(left_in2),
    _right_en(right_en),
    _right_in1(right_in1),
    _right_in2(right_in2),
    _corked(false),
    _pending(false),
    _freq_hz(1000)
{
    // WHAT:
    // Initialize all logical output channels to neutral/stop.
    // WHY:
    // If ArduPilot reads back an output before writing to it, it should see a
    // safe value. For a bidirectional motor output, 1500 us means stop.
    for (uint8_t i = 0; i < 16; i++) {
        _last_pwm[i] = PWM_STOP;
        _enabled[i] = false;
    }
}

void RCOOutput_PCA9685_L298N::init()
{
    // WHAT:
    // Configure the PCA9685 and immediately stop both motors.
    // WHY:
    // On boot, motor outputs must fail safe. Never assume the PCA9685 state is
    // harmless after power-up or after a previous program used it.
    init_pca9685();

    write_motor(true, PWM_STOP);
    write_motor(false, PWM_STOP);
}

void RCOOutput_PCA9685_L298N::set_freq(uint32_t chmask, uint16_t freq_hz)
{
    // WHAT:
    // Set the PCA9685 PWM frequency.
    // WHY:
    // PCA9685 has one shared frequency for all channels. We ignore chmask here
    // because ENA/ENB and INx are all on the same PCA9685 chip.
    // HOW:
    // Clamp to a practical PCA9685 range. For L298N speed control, 50 Hz works
    // but is audible and coarse; 500-1000 Hz is a reasonable experiment range.
    if (freq_hz < 50) {
        freq_hz = 50;
    }

    if (freq_hz > 1600) {
        freq_hz = 1600;
    }

    _freq_hz = freq_hz;
    set_pca9685_freq(_freq_hz);
}

uint16_t RCOOutput_PCA9685_L298N::get_freq(uint8_t chan)
{
    // All PCA9685 channels share the same frequency.
    return _freq_hz;
}

void RCOOutput_PCA9685_L298N::enable_ch(uint8_t chan)
{
    // WHAT:
    // Mark a logical ArduPilot output channel as active.
    // WHY:
    // ArduPilot commonly enables outputs before using them. We only act on
    // writes to enabled channels.
    if (chan >= 16) {
        return;
    }

    _enabled[chan] = true;
}

void RCOOutput_PCA9685_L298N::disable_ch(uint8_t chan)
{
    // WHAT:
    // Disable a logical output channel and stop the related motor if needed.
    // WHY:
    // Disabling a motor output should not leave the last duty cycle running.
    if (chan >= 16) {
        return;
    }
}

```

```

    }

    _enabled[chan] = false;

    if (chan == LEFT_AP_CH) {
        write_motor(true, PWM_STOP);
    } else if (chan == RIGHT_AP_CH) {
        write_motor(false, PWM_STOP);
    }
}

void RCOutput_PCA9685_L298N::write(uint8_t chan, uint16_t period_us)
{
    // WHAT:
    //   Main bridge from ArduPilot logical output to physical motor control.
    // WHY:
    //   ArduPilot already did the rover/skid-steer mixing. This backend must not
    //   decide turn-left or turn-right. It only obeys left/right motor commands.
    // HOW:
    //   channel 0/SERVO1 -> left motor
    //   channel 2/SERVO3 -> right motor
    if (chan >= 16) {
        return;
    }

    if (period_us < PWM_MIN) {
        period_us = PWM_MIN;
    } else if (period_us > PWM_MAX) {
        period_us = PWM_MAX;
    }

    _last_pwm[chan] = period_us;

    if (!_enabled[chan]) {
        return;
    }

    // If corked, store the latest value but delay physical output until push().
    if (_corked) {
        _pending = true;
        return;
    }

    if (chan == LEFT_AP_CH) {
        write_motor(true, period_us);
    } else if (chan == RIGHT_AP_CH) {
        write_motor(false, period_us);
    }
}

uint16_t RCOutput_PCA9685_L298N::read(uint8_t chan)
{
    if (chan >= 16) {
        return PWM_STOP;
    }

    return _last_pwm[chan];
}

void RCOutput_PCA9685_L298N::read(uint16_t *period_us, uint8_t len)
{
    if (period_us == nullptr) {
        return;
    }

    for (uint8_t i = 0; i < len; i++) {
        period_us[i] = read(i);
    }
}

void RCOutput_PCA9685_L298N::cork()
{
    // WHAT:
    //   Start a batch update.
    // WHY:
    //   If left and right motors are updated one at a time, the rover may briefly
    //   twitch. cork()/push() allows both sides to be committed together.
    _corked = true;
    _pending = false;
}

void RCOutput_PCA9685_L298N::push()
{
    // WHAT:
    //   Commit the latest left/right values after a batch update.
    // WHY:
    //   This is where the physical PCA9685 outputs are updated after cork().
    _corked = false;
}

```

```

    if (!_pending) {
        return;
    }

    if (_enabled[LEFT_AP_CH]) {
        write_motor(true, _last_pwm[LEFT_AP_CH]);
    }

    if (_enabled[RIGHT_AP_CH]) {
        write_motor(false, _last_pwm[RIGHT_AP_CH]);
    }

    _pending = false;
}

float RCOOutput_PCA9685_L298N::pwm_to_speed(uint16_t pwm_us)
{
    // WHAT:
    //   Convert RC-style microseconds into a normalized motor command.
    // WHY:
    //   L298N needs direction plus duty cycle, not 1000-2000 us servo pulses.
    // HOW:
    //   1000 us -> -1.0 full reverse
    //   1500 us ->  0.0 stop
    //   2000 us -> +1.0 full forward
    int16_t delta = static_cast<int16_t>(pwm_us) - static_cast<int16_t>(PWM_STOP);

    if (abs(delta) <= DEADZONE_US) {
        return 0.0f;
    }

    if (delta > 0) {
        return constrain_float(delta / 500.0f, 0.0f, 1.0f);
    }

    return constrain_float(delta / 500.0f, -1.0f, 0.0f);
}

void RCOOutput_PCA9685_L298N::write_motor(bool left, uint16_t pwm_us)
{
    // WHAT:
    //   Drive one physical motor side from one logical ArduPilot output value.
    // WHY:
    //   The L298N direction pins select polarity; ENA/ENB selects speed duty.
    // HOW:
    //   Forward: IN1=ON,  IN2=OFF, EN=duty
    //   Reverse: IN1=OFF, IN2=ON,  EN=duty
    //   Stop:    IN1=OFF, IN2=OFF, EN=0
    const float speed = pwm_to_speed(pwm_us);
    const float duty = fabsf(speed);

    const uint8_t en = left ? _left_en  : _right_en;
    const uint8_t in1 = left ? _left_in1 : _right_in1;
    const uint8_t in2 = left ? _left_in2 : _right_in2;

    if (speed > 0.0f) {
        pca_on(in1);
        pca_off(in2);
        pca_write_duty(en, duty);
    } else if (speed < 0.0f) {
        pca_off(in1);
        pca_on(in2);
        pca_write_duty(en, duty);
    } else {
        pca_write_duty(en, 0.0f);
        pca_off(in1);
        pca_off(in2);
    }
}

void RCOOutput_PCA9685_L298N::init_pca9685()
{
    // WHAT:
    //   Put PCA9685 into a known output mode.
    // WHY:
    //   We use the PCA9685 as both PWM generator and digital output expander.
    if (_dev == nullptr) {
        return;
    }

    AP_HAL::Semaphore *sem = _dev->get_semaphore();
    if (sem == nullptr || !sem->take(100)) {
        return;
    }
}

```

```

    // MODE1: auto-increment enabled.
    _dev->write_register(PCA9685_MODE1, 0x20);

    // MODE2: totem-pole output.
    _dev->write_register(PCA9685_MODE2, 0x04);

    sem->give();

    set_pca9685_freq(_freq_hz);
}

void RCOOutput_PCA9685_L298N::set_pca9685_freq(uint16_t freq_hz)
{
    // WHAT:
    //   Program the PCA9685 prescaler.
    // WHY:
    //   The chip derives output frequency from its oscillator and a prescale value.
    // NOTE:
    //   ArduPilot's existing PCA9685 backend uses an oscillator correction factor.
    //   This prototype uses the nominal 25 MHz value for readability.
    if (_dev == nullptr) {
        return;
    }

    const float oscillator_hz = 25000000.0f;

    uint8_t prescale = static_cast<uint8_t>(
        roundf(oscillator_hz / (4096.0f * freq_hz)) - 1.0f
    );

    AP_HAL::Semaphore *sem = _dev->get_semaphore();
    if (sem == nullptr || !sem->take(100)) {
        return;
    }

    uint8_t old_mode = 0;
    _dev->read_registers(PCA9685_MODE1, &old_mode, 1);

    uint8_t sleep_mode = (old_mode & 0x7F) | 0x10;

    _dev->write_register(PCA9685_MODE1, sleep_mode);
    _dev->write_register(PCA9685_PRESCALE, prescale);
    _dev->write_register(PCA9685_MODE1, old_mode);

    sem->give();

    hal.scheduler->delay(5);

    if (sem->take(100)) {
        // Restart + auto-increment.
        _dev->write_register(PCA9685_MODE1, old_mode | 0x80 | 0x20);
        sem->give();
    }
}

void RCOOutput_PCA9685_L298N::pca_write_channel(uint8_t channel,
                                                uint16_t on,
                                                uint16_t off)
{
    // WHAT:
    //   Write one PCA9685 channel's ON/OFF counter values.
    // WHY:
    //   Each PCA9685 output has 12-bit timing within a PWM cycle.
    // HOW:
    //   LEDn_ON_L/H and LEDn_OFF_L/H are four consecutive registers.
    if (_dev == nullptr || channel >= 16) {
        return;
    }

    const uint8_t reg = PCA9685_LED0_ON_L + 4 * channel;

    uint8_t data[5] = {
        reg,
        static_cast<uint8_t>(on & 0xFF),
        static_cast<uint8_t>((on >> 8) & 0x0F),
        static_cast<uint8_t>(off & 0xFF),
        static_cast<uint8_t>((off >> 8) & 0x0F)
    };

    AP_HAL::Semaphore *sem = _dev->get_semaphore();
    if (sem == nullptr || !sem->take(10)) {
        return;
    }

    _dev->transfer(data, sizeof(data), nullptr, 0);

    sem->give();
}

```

```

void RCOOutput_PCA9685_L298N::pca_write_duty(uint8_t channel, float duty)
{
    // WHAT:
    //   Express a channel output as a simple duty cycle from 0.0 to 1.0.
    // WHY:
    //   Motor speed uses variable duty. Direction pins use 0.0 or 1.0 duty.
    duty = constrain_float(duty, 0.0f, 1.0f);

    if (duty <= 0.0f) {
        pca_write_channel(channel, 0, 0);
        return;
    }

    if (duty >= 1.0f) {
        pca_write_channel(channel, 0, 4095);
        return;
    }

    const uint16_t off_count = static_cast<uint16_t>(duty * 4095.0f);
    pca_write_channel(channel, 0, off_count);
}

void RCOOutput_PCA9685_L298N::pca_on(uint8_t channel)
{
    // Digital ON implemented as 100% PWM duty.
    pca_write_duty(channel, 1.0f);
}

void RCOOutput_PCA9685_L298N::pca_off(uint8_t channel)
{
    // Digital OFF implemented as 0% PWM duty.
    pca_write_duty(channel, 0.0f);
}

```

6. Conceptual board integration

The class above is a backend class. To use it, it must be instantiated from the AP_HAL_Linux board setup and assigned as the board RCOOutput driver. The exact integration point can differ by ArduPilot branch and board subtype.

```

// Conceptual board integration example only.
// The exact file and constructor style may differ by ArduPilot branch.

#include "RCOutput_PCA9685_L298N.h"

// Acquire the PCA9685 I2C device from the Linux HAL I2C manager.
// Typical PCA9685 default address is 0x40.
static auto pca9685_dev = hal.i2c_mgr->get_device(1, 0x40);

static Linux::RCOutput_PCA9685_L298N rcoutDriver(
    pca9685_dev.get(),

    0, // PCA9685 CH0 -> L298N ENA
    1, // PCA9685 CH1 -> L298N IN1
    2, // PCA9685 CH2 -> L298N IN2

    3, // PCA9685 CH3 -> L298N ENB
    4, // PCA9685 CH4 -> L298N IN3
    5  // PCA9685 CH5 -> L298N IN4
);

// Then wire the HAL's rcout pointer to &rcoutDriver in the appropriate
// AP_HAL_Linux board subtype setup.

```

7. Rover parameters

```

SERVO1_FUNCTION = 73  # Throttle Left
SERVO3_FUNCTION = 74  # Throttle Right

SERVO1_MIN  = 1000
SERVO1_TRIM = 1500
SERVO1_MAX  = 2000

SERVO3_MIN  = 1000
SERVO3_TRIM = 1500
SERVO3_MAX  = 2000

```


8. Bench test plan

1. Disconnect motor power or lift the rover so wheels cannot touch the table.
2. Confirm the PCA9685 I2C address, normally 0x40 unless address jumpers are changed.
3. Command stop first: SERVO1=1500 and SERVO3=1500. ENA/ENB should be zero duty and motors stopped.
4. Command small forward values such as 1550. Confirm both motors rotate in the desired forward direction.
5. If one side is reversed, either swap the motor wires or swap the IN1/IN2 mapping for that side.
6. Command opposite values such as left=1300, right=1700. Confirm the rover spins. If spin direction is opposite to expectation, adjust motor polarity/mapping.
7. Only after direction and stop behavior are verified should you test with wheels on the ground.

9. Limitations and follow-up improvements

- Prototype status: this is intended as a teaching backend. It may need small API adjustments to match the exact ArduPilot branch you compile against.
- The PCA9685 frequency uses a nominal 25 MHz oscillator. ArduPilot's existing PCA9685 driver applies an oscillator correction factor; you may want to copy that detail later.
- The code writes multiple I2C transactions per motor command. Later you can optimize by combining direction and EN writes or caching unchanged outputs.
- The stop mode uses coast: EN=0 and direction pins off. If you want braking, use a deliberate L298N brake state, but test current and heat carefully.
- L298N is inefficient and can get hot. For real rover use, a modern MOSFET H-bridge is usually better.

10. References used

- ArduPilot AP_HAL RCOOutput interface: https://github.com/ArduPilot/ardupilot/blob/master/libraries/AP_HAL/RCOutput.h
- ArduPilot AP_HAL_Linux PCA9685 RCOOutput implementation: https://github.com/ArduPilot/ardupilot/blob/master/libraries/AP_HAL_Linux/RCOutput_PCA9685.cpp
- ArduPilot Linux porting notes, RCOOutput section: <https://github.com/ArduPilot/ardupilot/wiki/blob/master/dev/source/docs/ardupilot-on-linux-porting.rst>
- ArduPilot dev documentation, RC Input and Output: <https://ardupilot.org/dev/docs/learning-ardupilot-rc-input-output.html>